AI를 쓰는 기획자의 판단법

Q. 이 책을 읽고 무엇이 남아야 하나요?

저는 설명할 수 있는 실제 결정 한 건을 남기려 합니다. 무엇을 만들지, 누구의 어떤 일을 바꿀지, 지금은 어디까지 할지, 무엇을 보고 선택을 바꿀지입니다. AI 제품을 기획하는 PM과 서비스 기획자가 자기 업무를 가져와 쓰도록 구성했습니다. 긴 기획서를 완성하는 것보다 오늘 막힌 선택을 다음 행동으로 옮기는 데 초점을 둡니다.

기능 요청을 그대로 받아 적으면 만들기는 쉬워집니다. 하지만 그 기능 밖에 있는 해결책을 놓칠 수 있어요. 저는 고객이 최근에 겪은 일과 지금 버티는 방법을 먼저 보고, 사용자에게 좋은 일과 사업이 감당할 일을 함께 묻습니다. 충돌을 지우기보다 무엇을 감수할지 사람의 선택으로 남깁니다.

먼저 실제 결정 하나를 고르세요. "영상 편집자가 길이가 맞지 않는 한 구간을 다시 생성할 수 있게 할까?"처럼 대상과 선택이 보여야 합니다. 계정이나 개발 도구는 필요하지 않습니다. 텍스트 편집기와 공개해도 되는 업무 메모면 시작할 수 있어요. 자료가 없으면 확인할 질문을 남기는 선택도 가능합니다.

README.md에서 네 파일을 준비한 뒤 2쪽으로 가세요. 이미 결정이 분명하면 24쪽의 최소 경로로 바로 가도 됩니다. 끝에는 선택 이유, 포기한 것, 책임자와 7일 뒤 검토일을 저장하고 파일을 다시 엽니다. 책 속 예시는 내 프로젝트의 사실이나 진행 상태를 대신하지 않습니다.

내가 고른 결정 한 건이 네 문서로 이어집니다

01 고객의 근거	02 선택과 실행 경계	03 다음 선택
customer.md	prd.md / architecture.md	progress.md
최근 사건과 현재 대안	범위, 품질과 사람 책임	이유, 실제 결과와 재판단

지금 막힌 결정을 먼저 고릅니다

Q. 아이디어가 여러 개인데 어디부터 시작하나요?

저는 “이번 주에 답이 필요한 질문”으로 좁힙니다. 제품 전체를 평가하려 하면 조사도 개발도 끝이 없어져요. 요청한 기능을 만들지, 기존 대안으로 해결할지, 고객 확인을 먼저 할지 가운데 지금 고를 일을 적습니다. 결정하지 않을 때 생기는 손실도 함께 봅니다. 서두를 이유가 관성뿐이라면 이번 주에 멈추는 편이 나을 수 있습니다.

자기 프로젝트의 progress.md에 DEC-001을 만들고 질문, 사용자 결과, 사업 목표, 쓸 수 있는 시간과 비용을 적으세요. 비용 상한을 모르면 임의의 금액을 넣지 말고 누가 언제 정할지 씁니다. 고객에게 이미 한 약속, 사용 권한, 바꿀 수 없는 일정도 제약입니다. 데이터가 없다는 사실을 목표가 없다는 뜻으로 바꾸지는 않습니다.

Q. 첫 선택은 어떤 형태면 충분한가요?

“오늘은 한 구간의 수동 검토안을 비교하고 실제 생성은 보류한다”처럼 허용 행동이 보여야 합니다. 자료가 부족한데 전체 개발을 시작하는 것과, 자료를 얻을 작은 행동을 하는 것은 다릅니다. 현재의 선택지에 조사, 수동 처리, 축소, 중단도 넣어 보세요.

이때 owner와 review 날짜를 정합니다. owner는 문서 작성자가 아니라 결과를 보고 다음 선택을 확정할 사람입니다. 오늘부터 7일 뒤의 날짜를 직접 계산해 쓰고, 이번 주에 관찰할 행동을 한 가지 정하세요. 아직 행동이 일어나지 않았다면 actual 칸은 비워 둡니다. 이 한 절이 뒤의 조사와 설계를 어디까지 할지 정하는 기준이 됩니다.

제가 반복해서 쓰는 여덟 가지 원칙

Q. 질문이 많아질수록 기획이 좋아지나요?

저는 질문 수보다 선택이 달라지는지를 봅니다. 다음 원칙은 제가 기능 요구를 다시 정의하고, 품질과 비용을 따지고, 팀의 평가 의견을 모을 때 연결해 쓰는 기준입니다. 고객을 이해하는 JTBD, 전제를 되묻는 소크라테스 질문법, 진단과 일관된 행동을 연결하는 RumeIt의 전략 관점에 제 판단 방식을 더해 적용합니다.

1. 요청 기능과 고객 문제를 나눕니다. 기능 이름을 빼야 다른 해결책도 보입니다.
2. 최근 행동과 현재 대안을 봅니다. 미래의 호감만으로 전환을 기대하지 않습니다.
3. 목표, 제약, 실패 비용을 먼저 적습니다. 감당하지 못할 약속을 막기 위해서입니다.
4. 사용자 가치와 사업 가치를 함께 봅니다. 한쪽의 손실을 조용히 감추지 않습니다.
5. AI에게 반대 질문을 구합니다. 제 선택을 지지하는 말만 모으지 않습니다.
6. 포기한 것과 재개 조건을 적습니다. 같은 논의를 이유 없이 반복하지 않기 위해서입니다.
7. 사실, 맥락, 목적을 나눠 봅니다. 맞는 설명도 우리 고객에게는 쓸모없을 수 있습니다.
8. 결정 뒤에 실제 결과를 덧붙입니다. 결과를 알고 나서 과거 이유를 고쳐 쓰지 않습니다.

여덟 개를 외울 필요는 없어요. 오늘의 결정에서 가장 쉽게 생략할 기준을 하나 고르고 progress.md에 이유를 쓰세요. 비용을 아끼느라 품질 약속을 바꾸려 한다면 4번, 기능 요청을 곧장 명세로 옮겼다면 1번부터 돌아봅니다.

다섯 단계는 다시 돌아오기 위한 순서입니다

Q. 시장조사부터 수익화까지 한 번씩 끝내면 되나요?

저는 시장조사, 문제정의, 기능설계, 검증 GATE, 수익화를 하나의 고리로 봅니다. 시장조사에서는 고객의 최근 행동과 대안을 확인하고, 문제정의에서는 역할과 바꾸려는 결과를 좁힙니다. 기능설계에서는 포함과 제외, 사람과 AI의 책임을 정합니다. 검증 GATE에서는 지금 가진 근거로 허용할 실행을 고르고, 수익화에서는 고객의 선택과 지속 운영 조건을 봅니다.

수익화가 마지막이라고 원가를 마지막에 계산하지는 않습니다. 처음부터 시간과 비용의 제약을 두고, 실제 실행 전에 사용량과 오류 비용을 확인해요. 사내 업무라면 억지로 매출을 만들지 않고 예산과 운영 담당자의 부담으로 바꿔 묻습니다. 기술적으로 가능해도 고객이 바꿀 이유가 없으면 문제정의로 돌아갑니다.

단계	지금 답할 질문	주로 바뀌는 절
시장조사	무엇으로 버티고 왜 바꿀까?	customer.md의 현재 대안
문제정의	누구의 어떤 결과를 바꿀까?	customer.md의 문제, prd.md의 목표
기능설계	무엇을 맡기고 무엇을 뺄까?	prd.md의 범위, architecture.md의 처리
검증 GATE	어떤 실행까지 허용할까?	progress.md의 결정
수익화	계속 선택하고 운영할 조건은?	prd.md의 지표, progress.md의 리뷰

7일 뒤 결과가 기대와 다르면 가장 먼저 틀어진 가정으로 돌아갑니다. 전환이 없는데 기능을 더하는 것으로 답하지 말고 현재 대안이 충분했는지 보세요. 한 단계를 다시 읽는 것이 앞의 기록을 지우는 일은 아닙니다.

네 문서로 같은 결정의 이유를 따라갑니다

Q. 하나의 기획서를 네 파일로 나누는 이유가 뭔가요?

저는 바뀌는 이유가 다른 내용을 나눕니다. 고객에게 들은 사실, 사람이 확정한 제품 기준, 그 기준을 지킬 처리 구조, 실제 실행의 결과는 서로 다르게 바뀝니다. 이 넷을 한 문장으로 섞으면 AI의 해석이 고객 발언이 되고, 구현 편의가 품질 약속을 바꾸기 쉽습니다. 문서 수를 늘리기보다 역할을 고정하는 편이 다음 판단을 돕습니다.

customer.md에는 불편, 기대, 현재 대안, 출처와 가설을 둡니다. prd.md에는 목표, 포함과 제외, 품질 기준의 정본을 둡니다. architecture.md에는 입력과 처리, 코드 책임, 사람 검토와 오류 복구를 둡니다. progress.md에는 결정, 선택 이유, 실제 결과와 7일 리뷰를 시간순으로 덧붙입니다. 비교표와 질문 카드는 이 네 파일의 절로 저장합니다.

예를 들어 customer.md의 INPUT-01이 “한 구간의 길이가 맞지 않는다”는 연습 입력이면, prd.md는 이를 참조해 Q1 원본 보존을 정하고, architecture.md는 Q1을 참조해 원본과 후보를 분리합니다. progress.md의 DEC-001은 왜 이 범위를 골랐는지 남깁니다. INPUT은 자료 식별자이며 인터뷰가 있었다는 증거가 아닙니다.

품질 기준을 바꾸려면 사람이 prd.md의 해당 기준을 확정하고 progress.md에 변경 이유와 이전 결정을 참조합니다. architecture.md에서 임의로 허용 범위를 넓히지 않습니다. 기존 네 파일이 있다면 템플릿을 덮어쓰지 말고, 필요한 절과 참조만 병합하세요. 회사 양식이 우선이면 각 역할이 있는 위치를 네 파일에 적어 연결합니다.

시장조사는 최근의 한 사건에서 시작합니다

Q. 고객이 필요하다고 말했는데 더 물어야 하나요?

저는 필요하다는 말 다음에 최근에 어떤 일이 있었는지 묻습니다. 불편의 크기는 그때 한 행동과 포기한 것으로 읽을 수 있어요. "잘 쓰실까요?"보다 "마지막으로 길이가 맞지 않았을 때 어떻게 처리했나요?"가 지금의 선택을 바꿀 정보를 줍니다. 기억하지 못하는 날짜나 원인은 제가 대신 채우지 않습니다.

먼저 customer.md의 최근 사건 절에 대상, 발생 상황, 현재 처리, 기대 결과, 출처와 모르는 점을 적으세요. 실제 고객 메모가 있으면 허용된 내용만 붙이고, 없으면 "인터뷰 없음"과 확인하고 싶은 사건을 적습니다. 아래 채운 예는 독립 연습 입력입니다. 실제 고객 증거로 저장하지 마세요.

메타프롬프트 1, 행동을 되짚을 질문

입력 양식: 대상 [역할], 최근 사건 [있던 일 또는 미확인], 현재 처리 [방법], 기대 [결과]. 이 네 항목의 본문을 먼저 붙인 뒤 요청합니다.

> 위 사건 메모에서 [역할]의 실제 행동을 확인할 질문 하나를 골라줘. 모르는 사실은 채우지 마.

채운 실행예: 대상 영상 편집자, 사건 한 구간의 더빙이 길어짐, 현재 처리 수동 편집, 기대 해당 구간만 수정. 이 내용을 먼저 붙입니다.

> 위 사건 메모에서 영상 편집자의 실제 행동을 확인할 질문 하나를 골라줘. 모르는 사실은 채우지 마.

설명용 응답예는 "마지막으로 길어진 구간을 발견한 뒤, 어떤 순서로 수정했나요?"입니다. 실제 응답이 두 가지 행동을 한꺼번에 묻는다면 첫 행동으로 좁히세요. 사람은 이 질문이 결정을 바꿀지 고르고 customer.md의 다음 확인에 저장합니다. 답을 받기 전에는 고객의 처리 시간을 만들지 않습니다.

고객이 지금 쓰는 방법까지 비교합니다

Q. 비슷한 AI 제품만 조사하면 충분하지 않나요?

저는 수작업과 그대로 두기도 비교합니다. 기존 편집 도구로 충분하다면 전환하지 않을 수 있어요. 바꿀 때의 학습과 원본 손실 우려도 비용입니다. 경쟁 기능은 공식 자료나 직접 사용으로 확인하고 날짜를 남깁니다.

customer.md의 현재 대안에 대안, 얻는 결과, 불편, 전환 부담, 확인 출처를 적으세요. 미확인인 기능을 없다고 단정하지 않습니다. 연습에서는 수동 편집, 전체 재생성, 기존 결과 유지라는 세 대안을 놓고 비교합니다. 이 표는 고객이 실제로 세 방법을 썼다는 기록이 아닙니다.

메타프롬프트 2, 전환을 막는 조건

입력 양식: 현재 대안 [방법], 새 접근 [방법], 기대 결과 [결과], 전환 제약 [부담]. 먼저 네 항목을 붙입니다.

> 위 대안 비교에서 [현재 대안]을 계속 쓸 이유 중 내 선택을 바꿀 조건 하나를 짚어줘. 자료 밖 성능은 추정하지 마.

채운 실행예: 현재 수동 편집, 새 접근 한 구간 재시도, 기대 나머지 구간 보존, 제약 원본 손실 우려. 먼저 이 본문을 붙입니다.

> 위 대안 비교에서 수동 편집을 계속 쓸 이유 중 내 선택을 바꿀 조건 하나를 짚어줘. 자료 밖 성능은 추정하지 마.

설명용 응답예는 “수동 편집으로 충분히 고칠 수 있다면 재생성이 필요한가?”입니다. 사람이 어느 대안을 직접 확인할지 골라 progress.md에 적습니다. 비교 자료가 없다면 조사 행동을 고르면 됩니다. AI가 새 도구를 추천한 것만으로 전환 가치가 확인되지는 않습니다.

페널티 요청을 문제의 문장으로 바꿨습니다

Q. 고객이 요청한 기능을 왜 그대로 만들지 않았나요?

구인구직 서비스를 맡았을 때 출근 전 노쇼에 페널티를 주고 표시해 달라는 요구가 있었습니다. 저는 그 요청을 고객 문제로 다시 정의했고, 양쪽 모두 책임을 갖도록 전자계약 절차를 넣는 방향을 제안했습니다. 출근 일정을 잡으면 계약서를 작성하는 흐름을 추가한 경험입니다. 여기서 남기려는 것은 페널티보다 계약이 언제나 낫다는 답이 아닙니다.

요청이 이미 해결책의 이름이면 대안이 좁아집니다. 페널티를 어떻게 설계할지만 물으면 출근 전에 약속을 분명히 하는 방법은 보기 어려워져요. 저는 이 장면을 읽을 때 요청 기능, 끝내려는 일, 영향을 받는 다른 당사자를 나눕니다. 여러분의 과제에서는 절차를 추가한 뒤 약속 이행과 중도 이탈이 어떻게 달라지는지도 보세요. 선택의 이유를 실제 효과와 연결해야 다음 판단을 할 수 있습니다.

Q. 같은 질문을 제 업무에서는 어떻게 써 보나요?

cases/01-request-to-problem.md의 독립 실습 A를 여세요. 이 실습에는 과거 사례와 별도의 예약 업무 입력이 있습니다. 담당자가 "노쇼한 사람을 표시하자"고 요청했다는 조건을 받아, 요청 문장과 문제 가설을 나누고 실제 사건을 확인할 질문을 고릅니다. 답을 상상해서 인터뷰로 기록하지 않습니다.

사람이 바로 기능화, 절차 재검토, 사건 확인 가운데 하나를 선택하고 네 파일의 해당 절에 저장하면 실습 한 번이 끝납니다. 직접 피해가 급하고 이미 확인된 근거가 충분하다면 임시 방어 조치가 먼저일 수 있어요. 그때도 누가 불이익을 받는지, 언제 되돌릴지 적습니다. 사례의 선택을 복제하지 말고 자기 조건에서 바꿀 이유를 남기세요.

쓰는 사람과 돈을 결정하는 사람을 나눕니다

Q. 고객 문제를 한 문장으로 쓰면 역할도 정해지나요?

저는 사용자, 구매자, 운영자를 따로 적습니다. 편집자는 구간을 빨리 고치고 싶지만 구매자는 비용을 예측하고 싶을 수 있어요. 운영자는 실패한 작업을 복구해야 합니다. 셋을 "고객"으로 묶으면 한 사람에게 좋은 기능이 다른 사람에게 어떤 부담을 주는지 놓칩니다. 같은 사람이 여러 역할을 맡아도 역할별 기대는 나눠 봅니다.

customer.md의 역할과 문제에 누가, 어떤 상황에서, 지금 무엇으로 처리하고, 무엇 때문에 원하는 결과를 못 얻는지 적으세요. 기능 이름을 지워도 문제를 설명할 수 있어야 합니다. JTBD는 고객이 특정 상황에서 이루려는 진전을 이해하는 기존 관점입니다. 여기서는 새 AI 기능을 정당화하기보다 해결책 밖의 선택을 여는 데 씁니다.

메타프롬프트 3, 문제 문장의 빈칸

입력 양식: 상황 [발생 장면], 사용자 [역할], 구매자 [역할 또는 미확인], 운영자 [역할], 문제 [현재 결과]. 먼저 본문을 붙입니다.

> 위 문제 문장에서 [역할]의 기대와 다른 역할의 기대가 섞인 부분 하나를 찾아줘. 확인되지 않은 역할은 미확인으로 둬.

채운 실행예: 상황 더빙 구간 길이 불일치, 사용자 편집자, 구매자 미확인, 운영자 작업 관리자, 문제 전체 재작업 부담을 가정. 먼저 붙입니다.

> 위 문제 문장에서 편집자의 기대와 다른 역할의 기대가 섞인 부분 하나를 찾아줘. 확인되지 않은 역할은 미확인으로 둬.

설명용 응답예는 "편집자의 수정 편의와 구매자의 비용 예측을 나누어야 한다"입니다. 실제 자료에 구매자가 없으면 이름을 만들어 넣지 말고 확인할 질문을 남깁니다. 사람이 고친 문제 가설을 customer.md에, 이번에 다룰 목표만 prd.md에 옮깁니다.

가장 약한 가정을 먼저 흔들어 봅니다

Q. 반대 근거를 찾으면 아이디어를 진행하기 어려워지지 않나요?

저는 실패했을 때 가장 비싼 가정부터 봅니다. 한 구간만 재생성하면 고객이 쓰리라는 기대가 약한데, 모델 속도만 시험하면 핵심 불확실성은 남습니다. 반대 근거는 아이디어를 공격할 장식이 아니라 이번 선택을 바꿀 정보입니다. 가정이 틀려도 쉽게 되돌릴 수 있다면 작은 실험으로 확인할 수 있어요.

progress.md의 결정 절에 가장 약한 가정, 이를 지지하는 자료, 반대되는 자료, 사실이 달라지면 고를 대안을 적으세요. 반대 자료가 아직 없으면 "없음"보다 "찾지 못함, 확인 방법"이라고 씁니다. 소크라테스 질문법을 적용할 때도 질문을 끝없이 늘리기보다, 답이 달라지면 실행 범위가 달라질 질문을 고릅니다.

메타프롬프트 4, 결정을 바꿀 반증

입력 양식: 선택 [현재 안], 핵심 가정 [성립 조건], 가진 자료 [본문], 실패 비용 [영향]. 먼저 이 내용을 붙입니다.

> 위 결정에서 [핵심 가정]을 반박할 관찰 하나를 제안해줘. 실제 반대 근거가 있는 것처럼 쓰지 마.

채운 실행예: 선택 한 구간 재시도 검토, 가정 수동 편집보다 도움이 됨, 자료 INPUT-01만 있음, 실패 비용 검토와 생성 비용. 먼저 붙입니다.

> 위 결정에서 수동 편집보다 도움이 된다는 가정을 반박할 관찰 하나를 제안해줘. 실제 반대 근거가 있는 것처럼 쓰지 마.

설명용 응답예는 "수동 편집만으로 작업을 끝내는 장면을 관찰한다"입니다. 사람이 그 관찰이 가능한지 고릅니다. 고객 접근 권한이 없다면 담당자에게 접근 조건을 확인하고, 실제 생성 대신 문서 비교를 허용하는 결정을 남기세요. 빈칸을 줄이는 행동과 전체 개발 승인을 구분합니다.

사용자 가치와 사업 목표의 충돌을 적습니다

Q. 두 목표를 모두 만족하는 안이 없으면 어떻게 하나요?

저는 공통으로 좋아지는 지점을 먼저 찾고, 남은 충돌은 감내할 선택으로 적습니다. 사용자 문제를 찾는 단계부터 사업 방향도 함께 묻고, 만족과 사용 행동에 관한 지표를 수익과 운영 지표와 같이 보는 방식을 써 왔습니다. 지표의 개수를 맞추는 규칙보다 중요한 것은 한쪽이 좋아질 때 다른 쪽에서 무엇이 나빠지는지 읽는 일입니다.

영상 현지화에서는 작업자의 수정 부담을 줄이려다 재시도 비용이 늘 수 있습니다. 비용을 줄이려고 재시도를 제한하면 결과가 나빠지거나 대기가 길어질 수 있어요. "비용 최적화"라고만 쓰면 사용자가 무엇을 감수하는지 드러나지 않습니다. prd.md의 목표와 측정에 사용자 결과, 사업 결과, 지켜야 할 품질, 감내할 손실과 책임자를 함께 적으세요.

Q. 기준선이 없는데 목표 수치를 어떻게 정하나요?

없는 수치를 업계 평균으로 채우지 않습니다. 먼저 지금 처리의 관찰 단위와 수집 방법을 정합니다. 예를 들어 작업별 선택 구간, 재작업 이유, 실제 작업 시간과 대기, 비용을 같은 조건으로 모으도록 계획할 수 있어요. 수집 권한과 보관 범위도 정하고, 기록 전 목표값은 미확인으로 둡니다. 숫자를 정할 사람이 따로 있다면 그 사람과 확정 날짜를 적습니다.

품질 조건을 지키기 어려우면 사용자 기대를 조용히 낮추지 말고 범위 축소, 수동 처리, 보류를 비교합니다. 이번 주의 합의가 "원본과 허용 구간은 지키고 생성은 보류한다"여도 판단은 가능합니다. 실제 전환과 지불은 다음 관찰 대상으로 남기고, 문서가 채워졌다는 이유로 확인됐다고 쓰지 않습니다.

AI를 넣기 전에 네 가지 방법을 비교합니다

Q. AI 제품인데 AI를 쓰지 않는 선택도 하나요?

저는 문제를 없앨 수 있는지부터 봅니다. 불필요한 단계를 제거하면 생성 결과를 검수할 일도 없어집니다. 다음은 입력과 절차를 표준화해 흔들림을 줄일 수 있는지 봐요. 판단 기준이 명확하면 규칙 자동화가 적합합니다. 의미 해석과 문장 변형처럼 정답을 미리 모두 적기 어려운 부분에서 AI 증강을 검토합니다. AI 증강은 사람이 검토하고 결정할 수 있게 후보를 돕는 방식입니다.

방법	선택할 조건	영상 구간 문제에 적용	남는 위험
제거	단계가 결과에 기여하지 않음	필요 없는 전체 재작업을 없앨지 검토	필요한 수정까지 없애는지 확인
표준화	입력과 절차의 차이가 오류 원인	구간 시작과 끝, 요청 이유를 정형화	예외를 억지로 맞출 수 있음
규칙 자동화	허용과 금지를 명시 가능	선택 구간 밖 변경을 차단하는 규칙	잘못 정한 규칙의 반복
AI 증강	해석이나 변형이 필요하고 검토 가능	구간 번역과 더빙 후보 생성	의미 오류, 길이 불일치, 비용

네 가지는 한 제품 안에 함께 있을 수 있습니다. 구간 선택의 유효성은 코드 규칙으로 확인하고, 번역 후보는 모델로 만들며, 채택은 사람이 할 수 있어요. 모델에게 비용 한도나 게시 권한을 추론하게 하지 않습니다.

자기 과제의 처리 한 단계를 골라 네 방법의 적용 가능성과 실패 비용을 prd.md의 대안과 범위에 적으세요. AI가 필요하다는 이유가 남지 않으면 수동 경로를 택해도 됩니다. 선택한 방식을 architecture.md의 처리 책임과 연결하고, 선택하지 않은 방식의 이유를 progress.md에 남깁니다.

포함과 제외, 품질 기준의 정본을 정합니다

Q. “품질 좋게”라고 쓰면 왜 부족한가요?

저는 작업자가 무엇을 받아야 하는지와 실패했을 때의 행동으로 바꿉니다. “한 구간 재시도”에는 원본이 보존되는지, 다른 구간이 바뀌지 않는지, 후보를 누가 채택하는지가 포함돼야 해요. 속도나 자연스러움도 필요할 수 있지만 자료 없이 합격 수치를 만들지는 않습니다. 범위와 품질이 섞이면 기능을 줄이면서 약속도 함께 줄이기 쉽습니다.

prd.md의 대안과 범위 아래 포함/제외 필드와 품질 기준을 작성합니다. 예시의 Q1은 원본 보존, Q2는 선택 구간만 변경, Q3은 단가, 재시도 상한, 승인 확인 전 생성 보류입니다. 이 조건은 고객 조사 결과가 아닌 연습의 결정입니다. 자기 권한과 기대에 맞게 확정하세요.

메타프롬프트 5, 기준의 모호함

입력 양식: 목표 [결과], 포함 [행동], 제외 [행동과 이유], 품질 [조건과 확인법]. 이 절들을 먼저 붙입니다.

> 위 PRD에서 [품질 조건]의 합격과 보류를 가를 수 없는 표현 하나를 찾아줘. 새 기준값은 만들지 마.

채운 실행예: 목표 선택 구간 검토, 포함 수동 비교, 제외 자동 게시, 품질 Q1 원본 보존, Q2 구간 외 변경 금지, Q3 생성 보류. 먼저 붙입니다.

> 위 PRD에서 원본 보존의 합격과 보류를 가를 수 없는 표현 하나를 찾아줘. 새 기준값은 만들지 마.

설명용 응답예는 “원본을 무엇과 대조해 보존을 확인하는가?”입니다. 사람이 확인 방법을 채우고 품질 기준을 확정합니다. 기준을 바꾸면 progress.md에 이유를 덧붙인 뒤 architecture.md의 참조를 고칩니다. 다른 문서에서 정본을 조용히 바꾸지 않습니다.

코드 구조와 사람의 결정 위치를 나눕니다

Q. 개발 전인데 architecture.md가 필요한가요?

저는 누가 무엇을 처리하고 실패하면 어디로 돌아가는지부터 적습니다. 상세 기술 선택이 없어도 입력, 출력, 사람 검토, 중단과 복구는 정할 수 있어요. 먼저 작업자가 구간을 선택하고, 규칙이 범위를 확인하고, 승인된 경우에만 모델이 후보를 만들고, 사람이 채택하는 흐름을 적습니다. 실제 생성이 보류라면 그 앞의 수동 검토까지만 허용합니다.

개발 폴더 예시는 프로젝트 루트의 네 문서와 src/domain/, src/adapters/, tests/fixtures/입니다. domain은 구간, 권한, 상한의 규칙을 맡고 adapters는 모델 연결을 맡습니다. fixtures는 허용된 시험 입력을 둡니다. 폴더를 적었다고 코드가 구현된 것은 아닙니다. 사람 검토인 HITL은 폴더 이름이 아니라 선택 권한이 필요한 처리 지점에 표시합니다.

메타프롬프트 6, 책임의 빈틈

입력 양식: 입력 [자료], 처리 [순서와 담당], 출력 [받을 것], 기준 [PRD의 Q-ID], 실패 [복구]. 먼저 이 내용을 붙입니다.

> 위 처리 흐름에서 [결정]의 책임자가 빠진 지점 하나를 찾아줘. PRD의 허용 범위는 바꾸지 마.

채운 실행예: 구간 선택은 편집자, 범위 확인은 규칙, 생성은 승인 대기, 후보 채택은 미정, Q1부터 Q3 준수, 실패 시 원본 유지. 먼저 붙입니다.

> 위 처리 흐름에서 후보 채택 결정의 책임자가 빠진 지점 하나를 찾아줘. PRD의 허용 범위는 바꾸지 마.

설명용 응답예는 “후보 채택 담당자를 지정해야 한다”입니다. 사람이 역할과 권한을 확정하고 architecture.md에 저장합니다. 오류가 나면 원본으로 돌아가고 재시도를 자동 반복하지 않는 경로도 적으세요. 비용 상한이 없으면 연결 구현을 서두르기보다 승인 조건을 먼저 확인합니다.

문장이 맞는지와 실행해도 되는지를 나눠 봅니다

Q. AI가 쓴 기획서가 자연스러우면 무엇을 더 보나요?

저는 먼저 사실, 맥락, 목적을 따로 묻습니다. 사실은 출처와 수치가 맞는지, 맥락은 이 고객과 제약에 맞는지, 목적은 원하는 결과를 돕는지입니다. 사실이 맞는 시장 설명도 지금 고객의 선택을 설명하지 못할 수 있어요. 이 세 질문은 초안 검수이며 실제 고객에게 가치가 있다는 증거를 대신하지 않습니다.

그다음 빠진 가정, 과한 기능, 미확인 수치, 흐린 범위의 네 렌즈로 읽습니다. customer.md의 가설을 PRD가 사실처럼 썼는지, 구간 검토에 자동 게시가 추가됐는지, 비용 절감률이 근거 없이 생겼는지, 실패 때 누가 무엇을 멈추는지 보세요. 발견과 원문은 progress.md의 현재 DEC-ID 아래 검토 메모에 적고 처리 이유를 남깁니다.

Q. 검수를 마치면 바로 전체 개발을 시작하나요?

저는 고친 문서와 실행 허용을 구분합니다. 고객이 정말 이 접근을 택하는지, 현재 대안이 충분한지, 원가와 권한이 맞는지를 확인한 뒤 실행 범위를 고릅니다. 허용 선택은 전체 개발, 작은 시험, 추가 확인, 보류, 중단이 될 수 있어요. 이번 결정의 위험과 되돌릴 수 있는 정도에 맞춥니다.

자료가 부족하다고 모든 행동을 막을 필요는 없습니다. 허용된 자료로 수동 비교는 할 수 있지만 실제 고객 영상 전송은 못 할 수 있습니다. 작은 시험을 허용할 때도 입력, 책임자, 비용, 성공과 보류 조건을 정하세요. 초안의 문제를 고쳤다는 사실과 고객의 문제를 해결했다는 결론은 서로 다른 기록으로 남깁니다.

동료 검토와 고객 인터뷰는 다른 답을 줍니다

Q. 동료가 좋은 아이디어라고 했으면 근거가 되나요?

저는 어떤 질문에 대한 근거인지 나눕니다. 동료는 요구의 모순, 기술 난점, 운영 부담을 잘 찾을 수 있어요. 하지만 실제 고객이 최근에 어떻게 일했고 무엇 때문에 바꾸는지 대신 말해 주지는 못합니다. AI 페르소나의 답도 인터뷰가 아닙니다. 고객 접근이 어렵다면 모의 답변을 만들기보다 접근할 방법과 그전의 허용 행동을 정합니다.

customer.md의 다음 확인에 실제로 겪은 사건을 묻는 질문을 적으세요. 인터뷰에서는 미래 의향만 묻기보다 마지막 사건, 선택한 대안, 포기한 것, 전환을 망설인 이유를 순서대로 듣습니다. 불분명한 기억은 그대로 남깁니다. 동료 검토는 별도 표기하고 결과를 고객의 지불 의사로 옮기지 않습니다.

메타프롬프트 7, 유도 없는 질문

입력 양식: 확인할 가정 [문장], 질문 초안 [문장], 대상 [역할], 최근 사건 [상황 또는 미확인]. 먼저 본문을 붙입니다.

> 위 질문 초안을 [대상]의 최근 행동을 묻는 질문 하나로 고쳐줘. 원하는 답을 암시하지 마.

채운 실행예: 가정 구간 재시도가 필요함, 초안 “이 기능 있으면 편하겠죠?”, 대상 편집자, 최근 길이 불일치 사건은 미확인. 먼저 붙입니다.

> 위 질문 초안을 편집자의 최근 행동을 묻는 질문 하나로 고쳐줘. 원하는 답을 암시하지 마.

설명용 응답예는 “최근 더빙 길이가 맞지 않았던 때 어떻게 처리했나요?”입니다. 사람이 질문을 고르고 접근 동의를 확인한 뒤 실제로 묻습니다. 인터뷰가 없었다면 customer.md에는 “미실시”를 유지하고 progress.md에 다음 확인 담당과 날짜를 남깁니다.

가장 작게 확인할 일과 전체 개발을 구분합니다

Q. 작은 시험이면 무엇이든 해도 되나요?

저는 작다는 말을 기간이나 화면 수로만 판단하지 않습니다. 고객 자료를 외부로 보내거나 돈을 쓰고 게시하는 시험은 작아 보여도 책임이 큽니다. 시험 하나가 어떤 가정을 줄이는지, 누가 허용하는지, 잘못되면 무엇을 복구하는지부터 정해요. 수요를 확인하려는 시험과 모델이 길이를 맞출 수 있는지 보는 시험도 분리합니다.

progress.md에 시험할 가정과 허용 행동, 금지 행동, owner, 종료 조건을 적습니다. prd.md의 품질 기준이 시험에도 적용되는지 보고, 달라져야 한다면 사람이 범위를 먼저 확정합니다. 예시에서는 공개 가능한 입력으로 수동 비교를 하되 단가, 상한과 승인 전 실제 생성은 보류합니다. 문서 비교만으로 음질이나 지불 의향이 확인됐다고 쓰지 않습니다.

메타프롬프트 8, 가장 작은 확인

입력 양식: 가정 [문장], 자료 [가진 입력], 권한 [허용과 금지], 한도 [시간과 비용 또는 미확인]. 먼저 붙입니다.

> 위 조건에서 [가정]을 확인할 가장 작은 행동 하나를 제안해줘. 허용된 자료와 권한 밖으로 나가지 마.

채운 실행예: 가정 한 구간 수정 흐름을 이해할 수 있음, 자료 INPUT-01, 권한 수동 검토만, 생성 비용과 상한 미확인. 먼저 붙입니다.

> 위 조건에서 한 구간 수정 흐름을 이해할 수 있다는 가정을 확인할 가장 작은 행동 하나를 제안해줘. 허용된 자료와 권한 밖으로 나가지 마.

설명용 응답예는 “입력에서 후보 채택까지 종이 흐름을 한 번 따라간다”입니다. 사람이 채택하거나 범위를 고친 뒤 실제 관찰을 적습니다. 실행하지 못했다면 이유와 새 확인 일을 덧붙이세요. 실행하지 않은 계획을 시험 결과 칸에 복사하지 않습니다.

비용을 줄이는 안에서 품질과 대기를 물었습니다

Q. 반복 생성을 줄이면 비용 문제를 해결할 수 있지 않나요?

영상 AI를 논의할 때 길이 보정의 반복 비용이 문제였고, 반복 횟수와 자원 할당, 모델을 바꾸는 안이 나왔습니다. 저는 반복을 줄이면 번역 품질이 어떻게 달라지는지, 평균 재시도와 언어별 차이, 반복마다 의미 있는 개선이 있는지 물었습니다. 더 빨리 적용하면 품질을 확인할 시간이 줄고, 자원을 줄이면 대기가 늘 수 있다는 갈등이 있었습니다.

당시 유료 전환을 위해 다운로드를 제한하는 안도 봤습니다. 하지만 생성은 여전히 가능하므로 생성 비용에는 큰 영향이 없다고 판단했습니다. 제가 묻고 싶은 것은 "무엇을 제한했는가"보다 "어느 행동에서 원가가 생기는가"입니다. 다운로드, 생성, 재시도의 단위를 나눠야 전환을 유도할 안과 비용을 줄일 안을 혼동하지 않습니다.

Q. 이 판단을 독립적으로 연습하려면요?

cases/02-quality-and-cost.md의 실습 B에서 실제 기록과 별도의 입력으로 한 구간 재시도안을 검토합니다. 원본 보존과 선택 구간만 변경을 지키며 수동 검토를 할지, 추가 자료를 기다릴지 고르세요. 단가, 상한과 승인이 정해지기 전 실제 생성은 보류합니다. 한 구간이 항상 싸다는 결론 대신 비교할 처리량을 정합니다.

같은 파일의 추가 연습에서는 조건을 바꿔 봅니다. 수동 대안으로 충분하면 범위를 축소하고, 권한과 한도, 확인법이 생겨도 전체 개발보다 좁은 시험을 검토합니다. INPUT-02의 가정 관찰을 받아 REVIEW-001, DEC-002와 기준 변경까지 별도 연습본에 쓰세요. 자기 실제 결과와 미래 actual은 유지합니다.

사용량, 원가, 고객 가치의 단위를 맞춥니다

Q. 모델 단가만 알면 수익성을 계산할 수 있나요?

저는 고객이 가치를 받는 단위와 비용이 생기는 단위를 먼저 맞춥니다. 영상 한 건을 결제해도 비용은 구간 수, 언어, 길이, 재시도, 검토와 지원에서 생길 수 있어요. 평균 하나로 긴 작업과 실패가 많은 작업을 감추지 않습니다. 무료 사용량이 늘 때의 부담, 실패한 시도와 재작업도 계산 입력에 포함합니다.

prd.md의 운영 지속 조건에 사용 단위, 원가 항목, 수집 방법, 지불과 도입 근거를 적으세요. 확인한 단가는 확인 날짜와 공급자 자료를 함께 남깁니다. 단가, 상한과 사용량이 없으면 확정 이익률을 쓰지 않습니다. 고객의 지불 의향도 실제 구매나 도입 행동과 구분합니다. 사내 도구는 예산과 담당자의 지속 운영 여력으로 봅니다.

메타프롬프트 9, 계산 전에 빠진 입력

입력 양식: 가치 단위 [고객이 받는 결과], 사용 단위 [처리량], 비용 [확인값과 미확인], 지불 [실제 근거]. 먼저 붙입니다.

> 위 운영 메모에서 [선택]의 비용 비교를 막는 미확인 입력 하나를 짚어줘. 단가와 수요를 만들어 계산하지 마.

채운 실행예: 가치 구간 수정, 사용 한 구간, 비용 단가와 재시도 상한 미확인, 지불 실제 근거 없음. 먼저 붙입니다.

> 위 운영 메모에서 한 구간 재시도의 비용 비교를 막는 미확인 입력 하나를 짚어줘. 단가와 수요를 만들어 계산하지 마.

설명용 응답예는 “재시도를 포함한 실제 처리량이 필요하다”입니다. 사람이 그 입력의 확인 담당을 정하고 progress.md에 남깁니다. 비용을 알기 전 가격 약속이나 무제한 제공을 확정하지 마세요. 확인할 수 없다면 수동 경로와 범위 축소를 함께 비교합니다.

시작할 때 7일 뒤의 선택도 정합니다

Q. 결과가 나온 다음에 회고하면 늦나요?

저는 시작할 때 무엇을 관찰하고 어떻게 선택을 바꿀지 적습니다. 결과를 본 뒤 성공 기준을 고르면 우연한 반응을 성과로 읽기 쉬워요. 7일은 첫 검토 주기입니다. 장기 유지나 수익이 증명되는 보편 기간은 아닙니다. 이번 주에 볼 수 있는 행동과 더 기다려야 하는 결과를 나눕니다.

progress.md의 DEC-001에 review 날짜, 관찰 방법과 유지, 수정, 축소, 중단/보류, 추가 관찰의 조건을 적으세요. 기준선이 없으면 수집부터 계획합니다. 추가 관찰이면 확인 대상, 담당, 날짜와 그동안 보류할 실행 범위를 정합니다. 인터뷰를 못 했다면 접근 방법을 다시 고르고 actual은 예상으로 채우지 않습니다.

메타프롬프트 10, 재판단 조건

입력 양식: 결정 [선택], owner [책임자], review [날짜], 관찰 [행동과 방법], 제약 [이번 주 한계]. 먼저 본문을 붙입니다.

> 위 결정에서 [검토일]에 유지와 수정을 가를 관찰 조건 하나를 제안해줘. 예상 결과를 실제 관찰로 쓰지 마.

채운 실행예: 결정 수동 검토 진행과 생성 보류, owner 프로젝트 담당자, review 2026-11-17, 관찰 구간 선택과 채택 흐름, 제약 인터뷰 없음. 먼저 붙입니다.

> 위 결정에서 2026-11-17에 유지와 수정을 가를 관찰 조건 하나를 제안해줘. 예상 결과를 실제 관찰로 쓰지 마.

설명용 응답예는 “구간 밖 변경 위험을 수동 흐름에서 구별할 수 있었는가?”입니다. 사람이 조건을 확정합니다. 7일 뒤에는 새 항목으로 실제 관찰과 다음 결정을 덧붙이고 원래 항목을 참조하세요. 예상과 다르다는 이유로 당시 선택 이유를 고치지 않습니다.

개인 평가를 목표 중심의 우선순위로 연결합니다

Q. 사용성 문제를 많이 찾으면 개선 순서도 정해지나요?

담당 제품의 휴리스틱 평가에서 저는 개별 평가로 끝내지 않았습니다. 각자 평가한 뒤 서로의 이유를 이해하는 공감 평가를 거쳐 목표 중심으로 우선순위를 정하는 순서로 운영했습니다. 같은 화면을 보고도 무엇이 문제인지 다를 수 있어요. 많이 언급된 항목을 고르기보다 어떤 사용자 과정에 먼저 영향을 주는지 설명해야 합니다.

AI 제품에서는 기술을 드러내고 싶어질 수 있습니다. 저는 복잡성을 감추면서 사용자가 도구를 활용할 수 있도록 경험을 단순화하는 쪽을 봅니다. 기능을 더했는데 고객의 일이 복잡해졌다면 지금 흐름을 살펴야 해요. 개별 이유를 지우고 합의한 순서만 남기면 다음에 왜 그렇게 골랐는지 돌아보기 어렵습니다.

Q. 제 상황에서는 언제 같은 순서를 쓰나요?

cases/03-ux-priorities.md에서 구간 선택의 모호함과 옵션의 복잡함을 따로 평가한 뒤 이유를 설명하고, “원본을 지키며 수정 후보를 검토한다”는 목표로 첫 개선을 고릅니다. 목표가 공유되지만 개선 의견이 다를 때 이 순서가 유용합니다. 기준 위반이 이미 분명하면 위험을 막는 조치부터 정할 수 있어요.

progress.md의 현재 DEC-ID 아래 검토 메모에 개별 이유와 반대 의견을 적고, 선택과 포기에 합의한 순서와 적용 조건을 남깁니다. 숙련 사용자가 옵션을 잘 쓰거나 실제 관찰에서 다른 중단 지점이 보이면 순서를 다시 고르세요. 팀의 합의와 실제 고객 행동은 다르므로 확인 대상과 담당, 날짜를 남깁니다.

네 템플릿은 같은 결정에 필요한 절부터 채웁니다

Q. 빈칸을 모두 채운 다음 기획을 시작해야 하나요?

저는 하나의 결정에 필요한 절부터 씁니다. 새 프로젝트라면 templates/의 네 파일을 자신의 프로젝트 루트로 복사하세요. 먼저 progress.md에 결정 질문과 owner, review를 적고, customer.md에 현재 가진 자료와 가설을 나눕니다. 이어 prd.md에서 이번 목표, 포함과 제외, 품질을 확정하고 architecture.md에 처리와 사람 검토, 복구를 적습니다. 마지막으로 progress.md에 선택 이유와 다음 행동을 이어 씁니다.

각 파일에는 구체적 입력 설명이 있습니다. [역할]에는 "편집자"처럼 책임 있는 역할을 쓰고, [근거]에는 실제로 읽은 자료의 위치와 날짜를 씁니다. 자료가 없으면 미확인과 확인할 일을 적으면 됩니다. 작성하지 못한 칸을 AI가 그럴듯하게 채우게 하지 마세요. 회사 필수 양식이 있다면 그 구조를 유지하고 네 역할이 어느 절에 있는지 연결합니다.

Q. 기존 파일이 이미 있으면 어떻게 합치나요?

먼저 현재 문서를 읽고 이번 결정과 겹치는 절을 찾습니다. 같은 품질 기준이 있으면 새 기준을 옆에 복제하지 말고 prd.md의 정본 위치를 참조하세요. 서로 다르면 누가 어떤 근거로 바꿀지 선택하고 progress.md에 새 결정을 남긴 뒤 해당 절을 고칩니다. 기존의 실제 결과와 날짜를 예시의 상태로 초기화하지 않습니다.

복사 후 네 파일을 각각 저장하고 닫았다 다시 엽니다. customer.md의 자료 ID가 PRD의 문제와 연결되고, architecture.md가 같은 Q-ID를 가리키며, progress.md가 같은 DEC-ID의 선택을 설명하는지 읽어 보세요. 파일 수보다 같은 결정을 끝까지 따라갈 수 있는지가 완료 조건입니다.

채운 예시는 선택 구간의 검토까지만 갑니다

Q. 영상 현지화 예시에서는 무엇을 결정했나요?

examples/video-localization/customer.md부터 네 파일을 여세요. 실제 회사 기록과 별개의 연습 입력입니다. 인터뷰와 실제 시험 결과는 없고 코드도 구현되지 않았습니다. 작업자가 선택한 한 구간의 재시도안을 검토하되, 원본을 보존하고 허용 구간 밖을 바꾸지 않으며 자동 게시를 하지 않는 조건을 둡니다. 이는 연습 조건이며 고객이 이미 요구한 사실로 읽지 않습니다.

이 예시의 자료 ID는 INPUT-01입니다. 원본 기록이나 고객 증거처럼 EV-ID를 붙이지 않습니다. customer.md는 무엇이 주어진 입력이고 무엇이 가설인지 구분합니다. prd.md는 Q1 원본 보존, Q2 선택 구간만 변경, Q3 단가, 재시도 상한, 승인 확인 전 생성 보류를 정본으로 둡니다. architecture.md는 그 조건을 처리와 중단 지점에 연결합니다.

Q. 생성이 보류인데 실습은 무엇을 하나요?

원본과 후보를 구분할 수 있는지, 구간 선택부터 채택까지 수동으로 따라가며 누가 결정하는지 확인할 수 있습니다. 비용과 모델 단가, 재시도 상한의 수치, 실제 지불 의향은 미확인입니다. 한도와 권한이 정해지기 전 실제 생성은 보류하지만, 문서 비교와 질문 정리는 진행할 수 있어요. progress.md의 actual은 실행 전이라 비어 있습니다.

실습 B의 별도 연습본에 INPUT-02, REVIEW-001, DEC-002를 잇고 PRD와 처리 확인법을 고칩니다. 가정 관찰을 실제 결과로 쓰지 않습니다. 이미 생성하거나 인터뷰한 독자는 자기 결과를 유지하고 필요한 절만 병합하세요.

내 과제로 바로 가거나 수동으로 끝낼 수 있습니다

Q. 시간이 부족하면 어디까지 해야 하나요?

저는 2쪽에서 결정 하나를 고른 뒤 5쪽의 네 파일 역할을 읽고 필요한 질문 한두 개만 고릅니다. 요청 재정의는 6쪽과 9쪽, 품질과 비용은 13쪽과 19쪽에서 시작하세요. 실제 과제가 있으면 예시를 먼저 완주하지 않아도 됩니다. 자기 업무의 허용된 자료와 현재 제약을 가져옵니다.

최소 경로는 결정 선택, 자료와 가설 구분, 대안 비교, 사람의 선택, 허용 행동과 품질 확정, 저장과 재열기, 7일 리뷰 예약입니다. customer.md에는 대안과 근거, prd.md에는 범위와 기준, architecture.md에는 처리와 복구, progress.md에는 이유와 owner, review를 남깁니다. 네 파일을 닫았다 다시 열어 같은 결정을 설명할 수 있는지 보세요.

Q. AI 계정이 없거나 자료를 보낼 수 없다면요?

책의 질문에 직접 답하고 대안을 손으로 비교하면 됩니다. 자료가 없으면 확인할 대상과 담당, 날짜를 고르고 그동안 보류할 실행 범위를 적습니다. 잘못된 AI 해석은 고객 사실을 고치지 않고 버린 이유를 현재 DEC-ID 아래 검토 메모에 남깁니다. 수동 경로도 같은 기준으로 선택을 확정합니다.

첫 저장 뒤에는 두 번째 과제 메모에 같은 기준을 적용하세요. 실제 메모가 없으면 연습임을 표시합니다. 그대로 쓸 조건, 바꿀 조건, 사람의 선택과 이유를 progress.md 끝의 재사용 기준에 근거 DEC-ID와 함께 적고 다시 엽니다. AI 없이도 이 대조를 해야 첫 결정의 기준을 다른 과제에 재사용한 기록이 남습니다.

판단을 반복할 때 hplan으로 이어 갑니다

Q. 네 파일로 끝낼 수 있는데 왜 도구가 필요한가요?

저는 같은 질문을 반복할 때 과거의 제외 이유와 결과를 다시 꺼낼 수 있는지를 봅니다. 프로젝트가 늘면 비용과 근거, 이전에 접은 방향을 매번 기억하기 어렵습니다. hplan은 공개 자료에서 근거와 원가, 제외 이력의 확인, 결정 누적과 경험칙 재사용을 다룹니다. 오늘 도구 없이 남긴 결정을 그런 반복 흐름에 이어 쓰는 선택 경로입니다.

hplan-guide.md에서 공개 저장소와 한국어 README를 엽니다. 첫 입력으로는 네 파일의 관련 절, 결정 질문, 근거와 반대 근거, 선택지, 제약과 재개 조건을 준비하세요. 내부 자료를 보낼 권한이 없다면 먼저 공개 가능한 범위를 정합니다. 사람이 확정한 기준과 미확인인 항목을 함께 전달해야 도구의 제안과 실제 허용을 구별할 수 있습니다.

Q. 파일 이름만 바꾸면 도구 기록으로 쓸 수 있나요?

Markdown은 사람이 읽고 판단한 기록입니다. 도구가 쓰는 JSONL은 한 줄마다 정해진 필드를 가진 별도의 데이터입니다. progress.md를 확장자만 바꿔 넣지 않습니다. 현재 공식 문서의 입력 형식과 저장 위치를 확인하고 옮길 항목을 사람이 검토해야 해요. 제안된 판정이 실제 실행 권한이나 고객 약속이 되는 것도 아닙니다.

단발성 결정 하나라면 네 문서와 7일 리뷰만으로 충분할 수 있습니다. 설치와 연결 자체가 새 일이 된다면 지금은 기록을 유지하세요. 도구를 고르는 기준은 설치 성공보다 다음 결정에서 이전 이유를 다시 쓸 수 있는지입니다. 이 책의 핵심 경로는 설치 없이 여기까지 끝납니다.

새 과제와 새 대화에서는 조건부터 바꿉니다

Q. 잘 나온 프롬프트를 그대로 다시 쓰면 되나요?

저는 질문의 역할을 재사용하고 고객과 근거는 다시 확인합니다. 새 대화에는 관련 customer.md의 자료, prd.md의 확정 기준, architecture.md의 허용 처리, progress.md의 현재 결정을 먼저 전달하세요. 파일에 접근하지 못하면 필요한 본문을 직접 붙입니다. 이름만 언급했다고 읽었다고 보지 않습니다.

두 번째 과제가 예약 업무라면 영상 예시의 구간 조건 대신 예약 당사자, 현재 대처, 불이익과 되돌릴 조건을 넣습니다. AI를 쓰면 실제 응답을 자료와 대조하고, 수동이면 두 번째 메모에 같은 기준을 직접 적용합니다. 두 경로 모두 progress.md 끝의 재사용 기준에 근거 DEC-ID, 그대로 쓸 조건, 바꿀 조건, 선택과 이유를 저장하고 다시 엽니다. 실제 메모가 없다면 연습임을 표시하세요.

Q. 자료가 없는데 AI가 고객 발언을 만들어 줬다면요?

그 발언을 고객 근거에서 제외합니다. “편집자가 원한다”는 말이 어느 자료에 있는지 확인하고, 없으면 가설과 다음 질문으로 돌립니다. 페르소나 답을 여러 개 만들었다고 인터뷰 수가 늘지 않습니다. INPUT-01은 연습 자료 ID이며 고객 증거가 아닙니다.

새 선택이 이전과 다르면 원래 결정을 지우지 않고 새 DEC-ID로 이유를 남깁니다. 추가 관찰을 고르면 확인 대상, 담당, 날짜와 그동안 보류할 실행 범위도 씁니다. 저장 후 재열어 이전 예시의 상태와 실제 결과를 복제하지 않았는지 읽으세요. 같은 결론이 나와도 적용 조건을 설명해야 재사용입니다.

7일 뒤에는 이전 결정을 지우지 않고 이어 씁니다

Q. 점검표를 채우면 무엇을 확인할 수 있나요?

저는 decision-checklist.md의 질문으로 선택과 실행 범위가 연결되는지 봅니다. 이번 결정, 바꿀 결과, 근거와 가설, 가장 약한 가정, 현재 대안, 선택과 포기, 품질과 복구, owner와 review가 한 결정을 설명해야 해요. 체크 표시는 별도 영구 기록으로 쌓지 않고 progress.md의 결정 절을 확인하는 데 씁니다.

저장 후 네 문서를 다시 열어 같은 결정을 따라갑니다. customer.md의 자료가 PRD의 목표와 연결되는지, architecture.md의 처리가 PRD의 허용을 넘지 않는지, progress.md의 다음 행동이 실제 담당자에게 가능한지 읽으세요. 없는 근거는 확인할 일과 보류 범위를 고릅니다. 중요한 것은 빈칸이 전혀 없는 문서보다 빈칸 때문에 무엇을 하지 않을지 아는 판단입니다.

Q. 7일 뒤 실제로는 무엇을 적나요?

새 리뷰에 이전 DEC-ID, 관찰 기간, 한 일과 못 한 일, 자료와 예상의 차이를 씁니다. 다음 선택은 유지, 수정, 축소, 중단/보류, 추가 관찰입니다. 추가 관찰이면 확인 대상, 담당, 날짜와 그동안 보류할 실행 범위를 적으세요. 기준이 바뀌면 새 DEC-ID를 남긴 뒤 PRD 정본과 처리를 갱신합니다. 실습 B의 INPUT-02로 이 전환을 별도 연습할 수 있습니다.

이전 actual 칸을 소급해서 채우기보다 실제 결과를 새 리뷰 항목에 남깁니다. 오탈자 정정이 필요해도 정정 이유를 덧붙이면 당시 판단이 보존됩니다. 7일로 보기 어려운 재사용과 수익은 더 긴 관찰일을 잡으세요. 이번 기록은 다음 과제에서 조건을 붙여 쓸 경험칙의 재료가 됩니다.

공개 출처를 열고 다음 행동 하나를 정합니다

Q. 이 방법의 개념은 어디에서 더 읽을 수 있나요?

저는 기존 개념을 새 이름으로 주장하기보다 출처를 확인하고 자기 결정에 적용합니다. JTBD는 고객이 상황 속에서 이루려는 진전을 이해하는 관점입니다. 소크라테스 질문법은 전제와 이유를 되묻는 전통입니다. RumeIt의 전략 커널은 진단, 방향을 정하는 정책, 일관된 행동을 연결합니다. 제 실무 적용은 요청 기능을 다시 정의하고, 감내할 손실과 제외 이유를 적고, 실제 결과로 판단을 이어 가는 데 있습니다.

- hplan 공개 저장소, 한국어 README: 판단을 반복할 때 근거, 비용, 제외 이력과 결정 기록을 연결합니다.
- AI_PM 공개 가이드: PM 업무와 제품 전략을 확장해 읽는 자료입니다.
- Christensen Institute, Jobs to Be Done: 고객의 상황과 선택을 이해하는 이론입니다.
- Stanford Encyclopedia of Philosophy, Plato's Shorter Ethical Works: 소크라테스적 문답과 검토의 배경입니다.
- Richard RumeIt 공식 사이트: 전략 저작과 저자 자료를 찾는 출발점입니다.
- Nielsen Norman Group, 10 Usability Heuristics: 사용성 평가의 기존 원칙입니다.

Q. 책을 닫기 전에 무엇을 하면 되나요?

하네스는 AI의 작업 방향을 정하는 판단 체계, GATE는 지금 허용할 실행을 고르는 지점, HITL은 사람이 결정에 참여하는 지점입니다. append-only는 과거 이유를 지우지 않고 다음 결과를 덧붙이는 방식입니다. 용어를 외우기보다 내 결정에 어디가 해당하는지 표시하세요. 지금 progress.md를 열어 다음 행동 하나와 책임자, 날짜를 확인하고, 7일 뒤 읽을 수 있는 곳에 저장하면 됩니다.